

JUNIOR FULLSTACK ENGINEER — Ent-JFE-20/05/26

Duration: 8 hours (2 calendar days)

Stack: Next.js 14, Fastify, PostgreSQL, Prisma, TypeScript strict, Jest

Deploy: Vercel (frontend) + Railway/Render (backend) + Supabase (database)

The Problem

A seller receives 1,000 orders per day. Each order: amount (USD), customer, payment method. System must: (1) record every order immutably, (2) process payments (Stripe mock), (3) calculate fees (3%), (4) prevent double-payments, (5) handle refunds, (6) daily settlement, (7) maintain double-entry ledger (debits always = credits), (8) audit trail for compliance. If something breaks, replay the day and know exactly what happened.

You will build: event store, double-entry ledger, payment processor (mocked Stripe), settlement logic, read model projections, concurrency-safe API, seller dashboard.

What We're Looking For

- Deep architecture thinking. Not just "write code" but "why this design."
- Financial precision. Decimal(18,4), no rounding errors, ledger always balanced.
- Concurrency mastery. 1,000 concurrent orders = no corruption.
- Idempotency guarantee. Same idempotencyKey = same result.
- State machine discipline. Strict validation of transitions.
- Code originality. Your own work, not copy-pasted tutorials. We can tell.
- Working code. Tests pass, deployment works, numbers add up.
- Honest git history. Realistic work over 2 days.

Part A: Backend — Event Store + Ledger

Task A.1: Financial Event Store Schema

- EventLog: id, aggregateId, eventType (OrderCreated, PaymentProcessing, PaymentConfirmed, FeeCalculated, OrderShipped, OrderDelivered), payload (JSON with Decimal), version, timestamp (UTC), idempotencyKey (unique).
- Ledger: id, orderId (FK), account (order_balance|payment|fees|payout), debit (Decimal 18,4, nullable), credit (Decimal 18,4, nullable), timestamp.
- Constraint: EXACTLY ONE of debit OR credit is non-null. Unique(aggregateId, version), Unique(idempotencyKey). Indexes: (aggregateId, version), (eventType), (timestamp).
- Document: Why Decimal(18,4)? Why separate ledger table? Why idempotencyKey? Why version as integer?

Task A.2: Financial Event Service

- recordOrder(orderId, amount, idempotencyKey) → emit OrderCreated, create ledger: DEBIT order_balance (+amount), CREDIT order_pending (+amount). ATOMIC.

- `recordPayment(orderId, amount, stripeId, idempotencyKey)` → emit `PaymentConfirmed`, create ledger: DEBIT `payment_received` (+amount), CREDIT `order_balance` (-amount). IDEMPOTENT.
- `calculateFees(orderId, amount, idempotencyKey)` → emit `FeeCalculated`, create ledger: DEBIT `fees_owed` (+3%), CREDIT `payment_received` (-3%).
- `dailySettlement(date)` → emit `SettlementProcessed`, create ledger: DEBIT `seller_payout` (balance), CREDIT `payment_received` (-balance). IDEMPOTENT.
- `verifyLedgerBalance(orderId)` → `sum(debits) - sum(credits) MUST = 0`.
- Test: concurrent `recordPayment` calls = one succeeds, one gets `VersionConflict`.

Task A.3: Stripe Mock

- `processPayment(orderId, amount, customerId)` → call mock Stripe, get: `{chargeId, status}`. IDEMPOTENT.
- Error handling: `StripeError`, `CardDeclined`, etc.

Task A.4: API Routes

- `POST /orders`, `POST /orders/:id/pay`, `GET /orders/:id`, `GET /orders/:id/ledger`, `POST /settle`, `GET /verify-ledger/:id`.
- All amounts = Decimal strings. All mutations require `idempotencyKey`.

Task A.5: Tests

- 9+ tests: happy path, idempotency, ledger balance, Decimal precision, concurrent orders, settlement idempotency, version conflict, invalid transition, projection consistency.
- Test with 100 concurrent orders. Verify: all recorded, no duplicates, ledger balanced.

Part B: Frontend

- B.1: Order Status Card — Show order amount, fees, payout, payment status. Real-time updates.
- B.2: Ledger Audit Trail — Full transaction history with debits/credits, running balance.
- B.3: Mobile-first Tailwind design. TypeScript strict.

Part C: Advanced

- C.1: Concurrency Under Load — 1,000 orders in 10 seconds = all recorded, ledger balanced, payouts correct.
- C.2: Decimal Precision — Edge cases (1 USD, 999,999.99 USD, $10 * 0.03 = 0.30$). No rounding errors.
- C.3: Settlement Idempotency — Settle twice = same result.

Part D: Code Review

Identify 5+ bugs: race conditions, precision errors, ledger imbalance, idempotency bugs, state machine violations.

```
async recordPayment(orderId, amount, idempotencyKey) {  const order = await
db.order.findUnique({ where: { id: orderId } });  if (order.payment_received > 0)
throw new Error('Already paid');  const existing = await
db.financialEvent.findUnique({ where: { idempotencyKey } });  if (existing) return
```

```
existing;  const payment = await stripeAPI.charge(amount);  const event = await
db.financialEvent.create({    data: { aggregateId: orderId, eventType:
'PaymentConfirmed', payload: { amount, chargeId: payment.id }, idempotencyKey,
version: order.version + 1 }  });  await db.order.update({ where: { id: orderId },
data: { payment_received: amount } });  return event; }
```

Evaluation

- Event log append-only.
- Ledger always balanced (sum debits = sum credits).
- Idempotency prevents duplicates.
- Concurrency safe (1,000 concurrent orders work).
- Decimal precision exact.
- 9+ tests pass.
- Frontend accurate + real-time.
- Can explain every design choice.
- Live deployment handles stress.

Submission

- GitHub repository link
- Frontend deployment URL
- Backend API base URL
- README with: architecture diagram, ledger design, concurrency strategy
- docs/ARCHITECTURE.md, docs/CONCURRENCY.md, docs/FINANCIAL_RULES.md
- Realistic 2-day git history
- Submit with: Ent-JFE-20/05/26

After submission: we stress-test (1,000+ concurrent orders), verify ledger balances, discuss your architecture decisions and approach.